

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL 12
SOLID PRINCIPLE



Disusun Oleh:
Zakiati Latifa 105224033

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Program SIAKAD (Sistem Informasi Akademik) adalah sebuah simulasi sistem untuk pengisian Kartu Rencana Studi (KRS) para mahasiswa yang dibuat berdasarkan prinsip-prinsip desain perangkat lunak berorientasi objek. Sistem ini dikembangkan dengan menerapkan lima prinsip SOLID dan pola Strategi untuk menjaga agar struktur kode tetap bersih, modular, serta mudah untuk ditingkatkan. Sistem ini terdiri dari berbagai file Java, masing-masing dengan tanggung jawab tertentu. Terdapat empat antarmuka utama, yaitu `MataKuliah` dan `OperasiPraktikum` yang berfungsi sebagai kontrak untuk tipe-tipe mata kuliah, `krsRepository` yang berfungsi sebagai abstraksi untuk penyimpanan data, serta `StrategiKalkulasiUKT` yang menjadi abstraksi untuk perhitungan UKT yang menerapkan pola Strategi. Antarmuka `MataKuliah` diimplementasikan oleh tiga kelas, yaitu `MataKuliahTeori`, `MataKuliahKKN`, dan `MataKuliahPraktikum`, di mana khusus untuk `MataKuliahPraktikum` juga mengimplementasikan `OperasiPraktikum` karena memiliki kebutuhan tambahan terkait dengan alokasi asisten lab dan pengecekan peralatan. Untuk menghitung UKT, ada lima pendekatan yang berbeda sesuai dengan tipe mahasiswa, yaitu `uktReguler`, `uktMbkm`, `uktBidikmisi`, `uktKaryawan`, dan `uktInternasional`, masing-masing menerapkan `StrategiKalkulasiUKT` dengan rumus yang bervariasi. Penyimpanan data KRS dilakukan oleh dua kelas, yaitu `MySQLDatabaseConnection` dan `CloudNoSQLConnection`, yang keduanya mengikuti interface `krsRepository`, sehingga penyimpanan dapat dimodifikasi tanpa memengaruhi logika bisnis. Kelas `krsService` berfungsi sebagai pengatur yang mengoordinasikan proses KRS dari validasi prasyarat oleh `krsValidator`, perhitungan tagihan UKT, penyimpanan data ke repositori, hingga pencetakan draf PDF oleh `krsPdfPrinter`. Semua proses ini diinisiasi melalui kelas `main` yang juga bertanggung jawab untuk melakukan eksekusi program.

II. Variabel

No	Nama Variabel	Tipe data	Fungsi
1	repository	krsRepository	Untuk menyimpan referensi implementasi penyimpanan KRS (<code>MySQLDatabaseConnection</code> atau <code>CloudNoSQLConnection</code>)
2	validator	krsValidator	Untuk menyimpan referensi objek untuk memvalidasi prasyarat mata kuliah mahasiswa

3	pdfPrinter	krsPdfPrinter	Untuk menyimpan referensi objek untuk mencetak draf KRS dalam format PDF
4	tagihan	double	Untuk menyimpan hasil kalkulasi UKT mahasiswa berdasarkan jumlah SKS dan jenis UKT
5	idMahasiswa	String	Untuk menyimpan identitas unik mahasiswa yang digunakan dalam proses KRS dan penyimpanan data
6	dataKRS	Object	Untuk menyimpan data KRS mahasiswa yang akan disimpan ke repository
7	jumlahSks	int	Sebagai parameter yang menyimpan jumlah SKS mata kuliah sebagai dasar perhitungan UKT
8	database	krsRepository	Untuk menyimpan instance implementasi repository yang digunakan (CloudNoSQLConnection) pada kelas main

III. Constructor dan Method

Dalam sistem ini menggunakan interface dan metode sebagai berikut:

No	Nama Metode	Jenis Metode	Fungsi
1	MataKuliah	<i>Interface</i>	abstrak untuk semua jenis mata kuliah, mendefinisikan getNamaMataKuliah() dan getSks()
2	OperasiPraktikum	<i>Interface</i>	Interface khusus praktikum yang mendefinisikan alokasiAsistenLab() dan cekPeralatanPraktikum()
3	krsRepository	<i>Interface</i>	Abstraksi penyimpanan data KRS, mendefinisikan kontrak simpanDataKRS()
4	krsService	<i>Constructor</i>	Digunakan untuk menginisialisasi objek

5	StrategiKalkulasiUKT	<i>Interface</i>	Abstraksi Strategy Pattern untuk perhitungan UKT, mendefinisikan hitungUKT(int jumlahSks)
6	getNamaMataKuliah()	<i>Function</i>	Mengembalikan nama mata kuliah dalam bentuk String
7	getSks()	<i>Function</i>	Mengembalikan jumlah SKS mata kuliah dalam bentuk int
8	alokasiAsistenLab()	<i>Procedural</i>	Menampilkan proses alokasi asisten lab untuk mata kuliah praktikum
9	cekPeralatanPraktikum()	<i>Procedural</i>	Menampilkan proses pengecekan peralatan praktikum (PC dan server lab)
10	simpanDataKRS()	<i>Procedural</i>	Menyimpan data KRS mahasiswa dengan menggunakan parameter idMahasiswa dan dataKRS ke media penyimpanan (MySQL atau Cloud NoSQL)
11	hitungUKT()	<i>Function</i>	Menghitung dan mengembalikan tagihan UKT berdasarkan jumlah SKS dan kategori mahasiswa
12	prosesKRS()	<i>Procedural</i>	Mengkoordinasikan alur proses KRS: validasi, hitung UKT, simpan data, dan cetak PDF
13	validasiPrasyarat()	<i>Function</i>	Memvalidasi prasyarat mata kuliah yang akan diambil mahasiswa, mengembalikan boolean
14	cetakDrafPDF()	<i>Procedural</i>	Mencetak draf KRS mahasiswa dengan menggunakan idMahasiswa dalam format PDF
15	main()	<i>Procedural</i>	Eksekusi program, dengan menginisialisasi semua objek dan mensimulasikan proses KRS 5 mahasiswa

IV. Dokumentasi dan Pembahasan Code

Program SIAKAD dirancang ulang dengan menerapkan SOLID Principle sebagai berikut:

1. Single Responsibility Principle (SRP)

Dalam program SIAKAD menerapkan SRP dengan memisahkan tanggung jawab atas satu fungsionalitas spesifik ke dalam kelas kelas yang berbeda yaitu:

- a. Class `krsValidator` yang hanya bertanggung jawab sebagai memvalidasi prasyarat mata kuliah mahasiswa. Apabila saat logika validasi perlu diubah, perubahan hanya terjadi di kelas ini tanpa memengaruhi kelas lain.

```
1 public class krsValidator {
2     public boolean validasiPrasyarat(String idMahasiswa, MataKuliah mk) {
3         System.out.println(x: "Memvalidasi syarat mata kuliah");
4         return true;
5     }
6 }
```

- b. Class `krsPdfPrinter` yang hanya bertanggung jawab sebagai mencetak draf KRS dalam format PDF. Pemisahan ini memastikan perubahan format PDF tidak akan berdampak pada logika penyimpanan maupun validasi.

```
1 public class krsPdfPrinter {
2     public void cetakDrafPDF(String idMahasiswa) {
3         System.out.println(x: "Mencetak PDF dengan format kop surat terbaru");
4     }
5 }
6
```

- c. `krsRepository` merupakan interface yang hanya mendefinisikan kontrak penyimpanan data KRS. Dengan adanya interface ini, kelas `krsService` tidak perlu peduli apakah data disimpan ke MySQL atau Cloud NoSQL.

```
1 public interface krsRepository {
2     void simpanDataKRS(String idMahasiswa, Object dataKRS);
3 }
```

- d. Class `krsService` hanya mengkoordinasikan alur proses KRS. Kelas ini tidak menangani detail teknis validasi, penyimpanan, maupun pencetakan PDF semuanya dilakukan ke kelas yang bertanggung jawab masing-masing.

```
1 public class krsService {
2     private krsRepository repository;
3     private krsValidator validator;
4     private krsPdfPrinter pdfPrinter;
5
6     public krsService(krsRepository repository, krsValidator validator, krsPdfPrinter pdfPrinter) {
7         this.repository = repository;
8         this.validator = validator;
9         this.pdfPrinter = pdfPrinter;
10    }
11
12    public void prosesKRS(String idMahasiswa, MataKuliah mk, StrategiKalkulasiUKT tipeUKT) {
13        if (!validator.validasiPrasyarat(idMahasiswa, mk)) {
14            throw new RuntimeException(message: "Prasyarat tidak terpenuhi!");
15        }
16
17        double tagihan = tipeUKT.hitungUKT(mk.getSks());
18        System.out.println("Tagihan UKT: Rp " + tagihan);
19
20        repository.simpanDataKRS(idMahasiswa, mk);
21
22        pdfPrinter.cetakDrafPDF(idMahasiswa);
23    }
24 }
```

2. Open/Closed Principle (OCP)

Dalam program SIAKAD menerapkan OCP melalui interface StrategiKalkulasiUKT. Setiap kategori mahasiswa memiliki kelas UKT nya masing-masing tanpa mengubah kelas krsService

- a. Class uktReguler yang UKT dihitung dengan mengalikan jumlah SKS dengan Rp150.000. Kelas ini digunakan untuk mahasiswa reguler yang membayar UKT sesuai tarif standar per SKS. Jika tarif berubah, cukup ubah nilai di kelas ini tanpa menyentuh kelas lain.

```
1 public class uktReguler implements StrategiKalkulasiUKT {
2     @Override
3     public double hitungUKT(int jumlahSks) {
4         return jumlahSks * 150000;
5     }
6 }
```

- b. Class uktMbkkm yang UKT dihitung dengan mengalikan jumlah SKS dengan Rp100.000. Tarif yang lebih rendah ini mencerminkan keringanan yang diberikan kepada mahasiswa yang mengikuti program MBKM. Penambahan kelas ini tidak memerlukan perubahan apapun pada krsService.

```
1 public class uktMbkkm implements StrategiKalkulasiUKT {
2     @Override
3     public double hitungUKT(int jumlahSks) {
4         return jumlahSks * 100000;
5     }
6 }
```

- c. Class uktBidikmisi yang UKT selalu mengembalikan nilai 0 karena biaya ditanggung penuh oleh beasiswa. Kelas ini tetap mengimplementasikan interface StrategiKalkulasiUKT sehingga dapat digunakan secara seragam oleh krsService. Hal ini membuktikan OCP berjalan dengan baik karena penanganan kasus khusus tidak memerlukan kondisi if di kelas utama.

```
1 public class uktBidikmisi implements StrategiKalkulasiUKT {
2     @Override
3     public double hitungUKT(int jumlahSks) {
4         return 0;
5     }
6 }
```

- d. Class uktKaryawan yang UKT dihitung dengan mengalikan jumlah SKS dengan Rp200.000. Tarif yang lebih tinggi dibanding reguler mencerminkan kategori mahasiswa karyawan yang memiliki kemampuan finansial lebih. Kelas ini berdiri sendiri dan dapat dimodifikasi tanpa memengaruhi strategi UKT lainnya.

```

1 public class uktKaryawan implements StrategiKalkulasiUKT {
2     @Override
3     public double hitungUKT(int jumlahSks) {
4         return jumlahSks * 200000;
5     }
6 }

```

- e. Class uktInternasional yang UKT dihitung dengan mengalikan jumlah SKS dengan Rp500.000. Tarif tertinggi ini diterapkan untuk mahasiswa internasional sesuai kebijakan universitas. Seperti strategi lainnya, kelas ini cukup ditambahkan tanpa mengubah satu baris pun kode pada krsService.

```

1 public class uktInternasional implements StrategiKalkulasiUKT {
2     @Override
3     public double hitungUKT(int jumlahSks) {
4         return jumlahSks * 500000;
5     }
6 }

```

3. Liskov Substitution Principle(LSP)

Program SIAKAD ini mengimplementasikan pada hierarki subclassnya dengan tanpa mengubah programnya. Hal ini juga dilakukan dari kategori ukt mahasiswa yang mengimplementasikan StrategiKalkulasiUKT dan akan diterapkan juga oleh matakuliah.

- a. MataKuliahTeori implements MataKuliah kelas ini tidak dibebani method yang tidak relevan seperti alokasi asisten lab. Objek MataKuliahTeori dapat langsung dioper ke prosesKRS() dan diproses tanpa perlu mengubah logika apapun di krsService.

```

1 public class MataKuliahTeori implements MataKuliah {
2     @Override
3     public String getNamaMataKuliah() {
4         return "Algoritma Pemrograman";
5     }
6     @Override
7     public int getSks() {
8         return 3;
9     }
10 }

```

- b. MataKuliahKKN implements MataKuliah kelas ini hanya mengimplementasikan MataKuliah tanpa tambahan interface lain karena KKN tidak membutuhkan operasi lab. Pergantian objek dari MataKuliahTeori ke MataKuliahKKN pada prosesKRS() tidak menyebabkan error apapun, membuktikan LSP terpenuhi.

```

1  public class MataKuliahKKN implements MataKuliah {
2      @Override
3      public String getNamaMataKuliah() {
4          return "Kuliah Kerja Nyata (KKN)";
5      }
6      @Override
7      public int getSks() {
8          return 4;
9      }
10 }

```

- c. MataKuliahPraktikum implements MataKuliah, OperasiPraktikum Implementasi dua interface sekaligus ini tidak melanggar LSP karena MataKuliahPraktikum tetap memenuhi seluruh kontrak MataKuliah sehingga dapat menggantikan objek MataKuliah manapun di krsService.

```

1  public class MataKuliahPraktikum implements MataKuliah, OperasiPraktikum {
2      @Override
3      public String getNamaMataKuliah() {
4          return "Praktikum Algoritma";
5      }
6      @Override
7      public int getSks() {
8          return 1;
9      }
10     @Override
11     public void alokasiAsistenLab() {
12         System.out.println(x: "Mengalokasikan asisten lab");
13     }
14     @Override
15     public void cekPeralatanPraktikum() {
16         System.out.println(x: "Mengecek PC dan server lab");
17     }
18 }

```

4. Interface Segregation Principle(ISP)

Program SIAKAD memisahkan interface MataKuliah dan OperasiPraktikum

- a. MataKuliah yang mendefinisikan getNamaMataKuliah() dan getSks() yang relevan untuk semua jenis mata kuliah

```

1  public interface MataKuliah {
2      String getNamaMataKuliah();
3      int getSks();
4  }

```

- b. OperasiPraktikum mendefinisikan alokasiAsistenLab() dan cekPeralatanPraktikum() yang hanya relevan untuk mata kuliah praktikum


```

1 public interface OperasiPraktikum {
2     void alokasiAsistenLab();
3     void cekPeralatanPraktikum();
4 }

```

MataKuliahTeori dan MataKuliahKKN hanya mengimplementasikan MataKuliah, tidak dipaksa mengimplementasikan OperasiPraktikum. Sedangkan MataKuliahPraktikum mengimplementasikan keduanya karena memang membutuhkan keduanya

5. Dependency Inversion Principle(DIP)

Dalam program SIAKAD DIP diterapkan pada krsService yang bergantung pada interface krsRepository, bukan pada implementasi class biasa

```

1 public class krsService {
2     private krsRepository repository;
3     private krsValidator validator;
4     private krsPdfPrinter pdfPrinter;
5 }

```

krsRepository merupakan interface yang mendefinisikan kontrak abstrak penyimpanan data KRS melalui satu method yaitu simpanDataKRS(). Interface ini menjadi jembatan antara krsService sebagai modul tingkat tinggi dengan implementasi penyimpanan konkret di bawahnya. Dengan adanya interface ini, krsService hanya perlu mengetahui bahwa ada sesuatu yang bisa menyimpan data KRS, tanpa perlu tahu apakah itu MySQL, Cloud NoSQL, atau media penyimpanan lainnya.

```

1 public interface krsRepository {
2     void simpanDataKRS(String idMahasiswa, Object dataKRS);
3 }

```

Kelas ini juga mengimplementasikan krsRepository namun menyimpan data ke MySQL Database secara sinkron. Keberadaan dua implementasi repository ini membuktikan bahwa DIP berjalan dengan baik — krsService tidak peduli implementasi mana yang digunakan karena keduanya memenuhi kontrak yang sama. Untuk beralih dari Cloud NoSQL ke MySQL, cukup ganti satu baris di main tanpa menyentuh logika bisnis

```

1 public class MySQLDatabaseConnection implements krsRepository {
2     @Override
3     public void simpanDataKRS(String idMahasiswa, Object dataKRS) {
4         System.out.println(x: "Menyimpan ke MySQL Database...");
5     }
6 }

```

Kelas ini merupakan implementasi konkret dari interface krsRepository yang menyimpan data KRS ke infrastruktur Cloud NoSQL secara asinkron. Penggunaan asinkron dipilih agar sistem tetap responsif dan tidak down saat terjadi lonjakan pengisian KRS. Kelas ini dapat diganti dengan implementasi lain kapan saja tanpa mengubah satu baris pun kode pada krsService

```

1 public class CloudNoSQLConnection implements krsRepository {
2     @Override
3     public void simpanDataKRS(String idMahasiswa, Object dataKRS) {
4         System.out.println(x: "Menyimpan ke infrastruktur Cloud NoSQL secara asinkron agar tidak down...");
5     }
6 }

```

Pada kelas main, seluruh dependensi yang dibutuhkan krsService dibuat dan dioper dari luar melalui konstruktor. Dengan cara ini, krsService tidak perlu mengetahui implementasi konkret mana yang sedang digunakan. Kelas ini kemudian mensimulasikan proses KRS untuk lima mahasiswa dengan kombinasi mata kuliah dan kategori UKT yang berbeda-beda

```

1 public class Main {
2     Run | Debug
3     public static void main(String[] args) {
4         krsValidator validator = new krsValidator();
5         krsPdfPrinter pdfPrinter = new krsPdfPrinter();
6         krsRepository database = new CloudNoSQLConnection();
7
8         krsService krsSer = new krsService(database, validator, pdfPrinter);
9
10        System.out.println(x: "--- Proses KRS Mahasiswa A (MBKM) ---");
11        krsSer.prosesKRS(idMahasiswa: "MHS-001", new MataKuliahTeori(), new uktMbkM());
12
13        System.out.println(x: "\n--- Proses KRS Mahasiswa B (Reguler) ---");
14        krsSer.prosesKRS(idMahasiswa: "MHS-002", new MataKuliahPraktikum(), new uktReguler());
15
16        System.out.println(x: "\n--- Proses KRS Mahasiswa C (Bidikmisi) ---");
17        krsSer.prosesKRS(idMahasiswa: "MHS-003", new MataKuliahKKN(), new uktBidikmisi());
18
19        System.out.println(x: "\n--- Proses KRS Mahasiswa D (Karyawan) ---");
20        krsSer.prosesKRS(idMahasiswa: "MHS-004", new MataKuliahTeori(), new uktKaryawan());
21
22        System.out.println(x: "\n--- Proses KRS Mahasiswa E (Internasional) ---");
23        krsSer.prosesKRS(idMahasiswa: "MHS-005", new MataKuliahTeori(), new uktInternasional());
24    }
25 }

```

IV. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa penerapan prinsip SOLID dalam program SIAKAD terbukti menghasilkan arsitektur kode yang bersih, modular, dan mudah dikembangkan. Single Responsibility Principle memastikan setiap kelas hanya memiliki satu tanggung jawab sehingga perubahan pada satu fitur tidak berdampak pada fitur lain. Open/Closed Principle melalui interface StrategiKalkulasiUKT memungkinkan penambahan jenis UKT baru tanpa mengubah kode yang sudah ada.

Liskov Substitution Principle memastikan semua implementasi MataKuliah dapat digunakan secara bergantian pada krsService tanpa mengubah perilaku program. Interface Segregation Principle dengan memisahkan MataKuliah dan OperasiPraktikum mencegah kelas dipaksa mengimplementasikan method yang tidak relevan. Dependency Inversion Principle melalui injeksi dependensi pada konstruktor krsService membuat sistem sangat fleksibel dalam mengganti implementasi penyimpanan data.